Getting Started with Google Gemini API

Today's goal • 3/3 items

Module 1
Make Your First Request with the Gemini API

- Welcome to the Google Gemini for developers specialization
  Video • 2 min
- Introduction to getting started with the Gemini API
  Video • 2 min
- Power up your applications with AI
  Video • 2 min
- **Gemini API fundamentals**
  Reading • 20 min
- Your first Gemini API call
  Video • 5 min
- Build an AI text generator
  Practice Assignment • 30 min
- Why Did That Work? Understanding Your First API Call
  Dialogue • 15 min
- Knowledge check: Gemini fundamentals
  Practice Assignment • 15 min

Module 2
Model Selection and Parameters

Module 3
Built-in Tools and Structured Output

Gemini API fundamentals

0:47 / 10:31 2x

Getting started with the Gemini API

Building applications with artificial intelligence requires a solid understanding of how these systems interpret input, process information, and generate responses. Google's Gemini API is a powerful tool that enables your applications to deliver intelligent, creative, and helpful interactions.

The Gemini API allows developers to connect their software directly with Google's most advanced generative models. It can be used to build chatbots, automate customer interactions, summarize complex documents, or generate creative content. The same system that powers Google's own AI experiences can now be integrated into your custom products and workflows, giving teams access to state-of-the-art reasoning and multimodal capabilities. Whether you are developing a prototype or a large-scale enterprise solution, understanding how the Gemini API operates helps you design applications that are efficient, secure, and contextually aware.

In this reading, you'll be introduced to the fundamental concepts you need to get started. By understanding the core ideas of tokens, context windows, rate limits, and secure authentication, you will build a strong foundation for creating powerful and reliable AI-powered applications.

Choosing your development environment: Google AI Studio vs. the Gemini API

Before writing any code, it's important to understand the two main ways for developers to build with Google's Gemini models: Google AI Studio and the Gemini API.

Google AI Studio provides a simple, visual interface that lets you experiment with Gemini models without writing code. It's ideal for quick exploration, testing prompts, and building prototypes. For instance, a marketing professional can use it to generate multiple headline variations, or a student can use it to brainstorm ideas.

The Gemini API, on the other hand, allows developers to integrate Gemini's capabilities directly into their own applications. It offers full flexibility and control for building custom AI solutions within websites, mobile apps, or enterprise systems. The API uses a set of rules and tools that enable your software to communicate with the Gemini models.

While AI Studio is best suited for experimentation and learning, the API is designed for production: creating scalable, reliable applications for real users. This course focuses on using the Gemini API to develop these kinds of solutions. The shift to using the robust Gemini API for production is excellent for vibe coding, as it ensures the initial conceptual momentum and "good feeling" of an experimental feature, and it can be translated smoothly and reliably into a stable, user-facing application without losing the original creative spark.

Understanding the core concepts of the API

To use the API effectively, you need to grasp a few foundational concepts that govern its cost, capacity, and performance.

Tokens and the context window

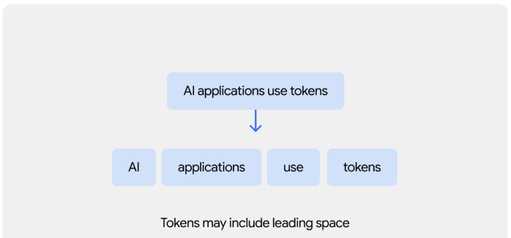
The most basic unit of language for the Gemini API is the "token." A token can be a piece of a word, a whole word, or even punctuation. The model does not see text the way humans do, as a series of letters and sentences. Instead, it breaks down all input and output text into a list of tokens. For example, the simple sentence "The cat sat." might be broken into four tokens: "The", " cat", " sat", and ".". A more complex word like "unforgettable" might be broken into three tokens: "un", "forget", and "table".

Understanding tokens is fundamental for two very important reasons: cost and capacity. First, you are typically billed based on the number of tokens you send to the model (your input) and the number of tokens the model sends back (its output). Being aware of how your text translates into tokens helps you estimate and control the cost of your application. Second, every model has a limit to how many tokens it can process at one time. This limit is known as the context window.

The context window functions as the model's active processing limit. It defines the total number of tokens that the model can "see" at any given moment. If the conversation or text exceeds this limit, the model can no longer reference earlier input. This has a huge impact on how you design your application.

Imagine you are building a customer service chatbot. A customer, Javier, starts a conversation: "Hi, my order #12345 has not arrived yet." If the context window is large enough, the model retains context about order #12345 when Javier later asks, "Can you check its status?" and can provide a relevant update. However, if the context window is too small, the second question might push the first message out of its "memory." The model would no longer know the order number and might respond with, "I'm sorry, I don't have an order number. Could you please provide it?" This creates a frustrating experience. A larger context window allows for longer, more coherent conversations, but it also means the model has to process more tokens, which can increase cost and response time.

How tokens work



Rate limits

As your application becomes more popular, you will encounter another important concept: rate limits. A rate limit is a rule that controls how many requests you can send to the API in a certain amount of time. For example, the free tier of the Gemini API might allow 60 requests per minute. Rate limits are measured in three dimensions: Requests per minute (RPM), Tokens per minute (input) (TPM), and Requests per day (RPD).

These limits are not there to slow you down. They are essential for maintaining a service that is stable, reliable, and fair for all users. They prevent any single application from accidentally or intentionally overwhelming the system, which could cause it to slow down or crash for everyone. If your app suddenly gets thousands of users and starts receiving "429: Too Many Requests" errors, it means you are hitting the rate limit. This is a normal part of growth, signaling that it is time to upgrade to a paid plan with higher limits or redesign your application to manage its requests more efficiently.

Making your first API call

Once you understand the core concepts, you can connect your application to the API. This involves securing an API key, using a Software Development Kit (SDK), and handling the model's response.

Securing your API key

To connect to the API, you need a special password called an API key. An API key is a unique string of letters and numbers that identifies your application and proves to Google that you have permission to use the service. You must keep your API key secret. If someone else gets your key, they could make requests on your behalf, which could cost you money or be used for malicious purposes.

To get a Gemini API key, you simply need to sign in to Google AI Studio and click the "Create API key" button, which instantly generates the unique string.

Never write your API key directly in your code (a practice known as "hard-coding"). The best practice is to store your key in an environment variable. This is a variable stored on your computer's operating system, outside of your code, where your application can securely access it when needed.

Using an SDK to send a request

With your API key securely stored, you are ready to make your first request. The easiest way to do this is by using an SDK. An SDK is a library of pre-written code that simplifies interaction with an API.

For a developer using Python, the process involves installing the library (`pip install google-genai`), configuring it with your API key, selecting a model, and calling the `generate_content` function with your prompt.

```
from google import genai

# Setting the API key as an environment variable

export GEMINI_API_KEY=<YOUR_API_KEY_HERE>


# Configure the API with your key (stored as an environment variable)
genai.configure(api_key="YOUR_API_KEY")


# Send a prompt to the model
response = client.models.generate_content(
model="gemini-3-pro-preview",
contents="write a short poem about the ocean."
)

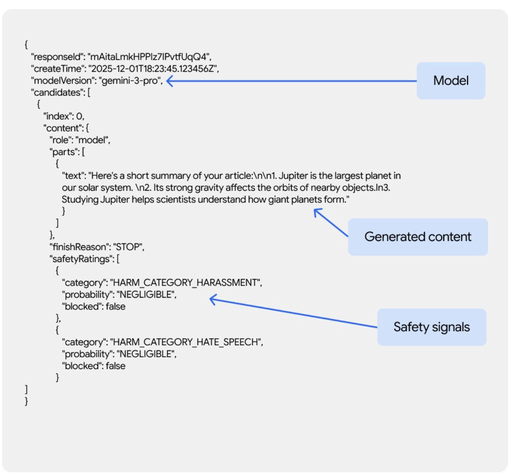
# Print the model's response
print(response.text)
```

Handling responses and errors

The API sends back a structured response object, which is a container holding the generated text and other useful information. In the code above, response.text accesses the actual poem. The object might also contain metadata about safety ratings or why the model stopped generating text. Learning to parse this object to extract the information you need is a fundamental skill.

Of course, things do not always go perfectly. You might have a typo in your API key or exceed your rate limit. When this happens, the API will send back an error. A robust application prepares for this using error handling techniques, like a try...except block in Python. This allows your program to "catch" an error and run fallback code, like printing a helpful message, instead of crashing.

Example: Handling responses and errors



Key takeaways

- **AI Studio vs. API:** AI Studio is a code-free tool for rapid prototyping and experimentation, while the API is used to integrate Gemini models directly into production applications.
- **Tokens:** Text is broken down into tokens, which are the basis for cost and capacity calculations.
- **Context window:** This defines how much input and output text the model can process at one time, measured in tokens. Balancing the size of the context window is a key design consideration for managing conversation quality, cost, and speed.
- **Rate limits:** These are measured across three dimensions: Requests per minute (RPM), Tokens per minute (input) (TPM), and Requests per day (RPD). These are caps on the number of requests you can make in a given time. They are vital for service stability and signal when it is time to scale your application.
- **Secure authentication:** Always protect your API key by storing it as an environment variable, never by hard-coding it directly into your application. It is recommended that you use the environment variable GEMINI_API_KEY, as this will be picked up automatically by the client when using one of the Gemini API libraries.
- **SDKs and error handling:** Use an SDK to simplify API calls, and implement error handling to build robust, user-friendly applications that can manage unexpected issues.

Go to next item ✓ Completed